I302 - Aprendizaje Automático y Aprendizaje Profundo

Trabajo Práctico 4: Aprendizaje No-Supervisado

Ana Paula Tissera

31 de mayo de 2025

Ingeniería en Inteligencia Artificial

1. Clustering de datos

1.1. Introducción

La presente sección aborda el problema de clustering, cuyo objetivo es identificar grupos de datos similares en un conjunto no etiquetado, de modo que las observaciones dentro de cada grupo compartan características comunes y difieran de las de otros grupos. Para ello, usamos el dataset provisto en el archivo `clustering.csv` que contiene muestras bidimensionales en $\mathbb{R}^2$, donde cada punto está definido por sus coordenadas (x_i, y_i). A partir de esta entrada, la tarea consiste en asignar a cada punto una etiqueta de cluster y determinar los parámetros que describen cada grupo (por ejemplo, centroides o distribuciones de densidad). Con este fin, se van a comparar varias técnicas de aprendizaje no supervisado con el fin de evaluar qué tan bien cada una logra reflejar la estructura interna del conjunto de datos.

1.2. Métodos

1.2.1. K-means con inicialización K-means++

El algoritmo K-means busca particionar el conjunto de N puntos $\{x_i\}_{i=1}^N \subset \mathbb{R}^2$ en K clusters minimizando la suma de las distancias al cuadrado dentro de cada cluster:

$$J(\{c_i\}, \{\mu_k\}) = \sum_{i=1}^{N} \left\| x_i - \mu_{c_i} \right\|^2, \tag{1}$$

donde $c_i \in \{1, \ldots, K\}$ es la etiqueta de cluster de x_i y $\mu_k \in \mathbb{R}^2$ el centroide del cluster k. Primero se inicializan los centroides mediante K-means++:

1. Elegir μ_1 al azar entre los puntos.
2. Para cada x_i, calcular $D(x_i) = \min_{j<k} \|x_i - \mu_j\|^2$.
3. Seleccionar el siguiente centroide con probabilidad proporcional a $D(x_i)$.
4. Repetir hasta obtener K centroides.

A continuación, en cada iteración se asigna cada x_i al cluster más cercano:

$c_i = \arg\min_k \|x_i - \mu_k\|^2$, y luego se actualizan los centroides como la media de los puntos asignados:

$$\mu_k = \frac{1}{N_k} \sum_{i:c_i=k} x_i, \quad N_k = \#\{i : c_i = k\}. \tag{2}$$

El proceso converge cuando las asignaciones $\{c_i\}$ dejan de cambiar.

1.2.2. Modelos de mezcla Gaussiana (GMM)

Un modelo de mezcla Gaussiana representa la densidad de los datos como combinación convexa de K normales:

$$p(x_i) = \sum_{k=1}^{K} \pi_k \mathcal{N}\big(x_i \mid \mu_k, \Sigma_k\big),$$

con pesos π_k ($\sum_k \pi_k = 1$), medias $\mu_k \in \mathbb{R}^2$ y covarianzas $\Sigma_k \in \mathbb{R}^{2\times 2}$. Se ajustan $\{\pi_k, \mu_k, \Sigma_k\}$ mediante Expectation–Maximization (EM):

- *E-step:* calcular responsabilidades

$$r_{ik} = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}. \tag{3}$$

- *M-step:* actualizar parámetros

$$\pi_k = \frac{1}{N}\sum_{i=1}^{N} r_{ik}, \quad \mu_k = \frac{\sum_{i=1}^{N} r_{ik}\, x_i}{\sum_{i=1}^{N} r_{ik}}, \quad \Sigma_k = \frac{\sum_{i=1}^{N} r_{ik}(x_i-\mu_k)(x_i-\mu_k)^T}{\sum_{i=1}^{N} r_{ik}}. \tag{4}$$

Se itera hasta que la mejora de la log-verosimilitud $\mathcal{L} = \sum_{i=1}^{N} \log p(x_i)$ cae por debajo de un umbral.

1.2.3. Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

DBSCAN agrupa puntos según densidad local sin requerir un número fijo de clusters. Dados dos hiperparámetros, el radio ε y el umbral *minPts*, se define el *ε-vecindario* de cada punto:

$$N_\varepsilon(x_i) = \{\, x_j : \|x_j - x_i\| \le \varepsilon\}.$$

A partir de este vecindario clasificamos:

- **Core point**: si $|N_\varepsilon(x_i)| \ge minPts$, es decir, tiene suficientes vecinos densos.
- **Border point**: si no es core, pero está en el ε-vecindario de al menos un core.
- **Noise (outlier)**: si no es ni core ni border, se marca como ruido.

Para formar los clusters, DBSCAN recorre cada core no asignado y expande su grupo agregando todos los puntos de su ε-vecindario; a su vez, para cada punto agregado que también sea core, se añaden recursivamente sus vecinos, hasta que se termine la densidad alcanzable. Así, cada cluster queda compuesto por un conjunto conectado de cores y sus fronteras, mientras que los noise points quedan fuera. Esta definición captura formas arbitrarias y separa ruido sin asumir formas convexas ni varianzas uniformes.

1.2.4. Conjunto de datos y preprocesamiento

Para los tres métodos se usó el mismo conjunto de datos, que contiene $N = 4999$ puntos en $\mathbb{R}^2$ sin etiquetar. Al tratarse de un problema no supervisado, no se dividió en subconjuntos de entrenamiento/validación/prueba.

En K-means y GMM se trabajó directamente sobre las coordenadas originales $\{x_i\}_{i=1}^{N}$. En DBSCAN, en cambio, antes de la clustering se aplicó normalización Z-score: $X_n = \frac{X-\mu}{\sigma}$, donde μ y σ son la media y la desviación estándar de cada dimensión en todo el dataset.

1.2.5. Hiperparámetros y selección

Para elegir el número óptimo de clusters en K-means, se analizó la inercia $L(K)$ (Sección 1.2.6), es decir, la suma de las distancias cuadráticas de cada punto x_i a su centroide μ_{c_i}, en función de K. Definimos las reducciones absolutas

$$D_k = L(k-1) - L(k), \quad k = 2, \ldots, K_{\text{máx}},$$

y su proporción relativa respecto a la primera caída D_2: $r_k = \frac{D_k}{D_2}$. Dependiendo de la pendiente de cambio y del nivel de detalle que se desea captar, se fija un umbral α y se elige el primer k tal que $r_k < \alpha$.

Además de este criterio de "ganancias decrecientes" sobre la curva de inercia, se usó la inicialización K-means++ (1) y los valores por defecto de la clase `KMeans` (en `models.py`): `n_init` $= 10$, máx $_iter = 300$, tol $= 10^{-4}$, `random_state` $= 42$.

En el modelo de Mezclas de Gaussianas (GMM), se mantuvo el mismo K, la misma tolerancia, número máximo de iteraciones y semilla, inicializando medias, pesos y covarianzas desde una corrida previa de K-means y agregando una regularización reg_covar $= 10^{-6}$ para asegurar la invertibilidad de las matrices de covarianza.

Para DBSCAN se exploró una malla de parámetros $\varepsilon \in [0, 0{,}25]$ (40 valores) y $minPts \in [30, 200]$ (40 valores). Inicialmente, mediante visualización de dos mapas —número de clusters detectados y proporción de puntos etiquetados como ruido— se pudieron inferir las combinaciones que separaba densamente los grupos reales sin generar demasiado ruido. Además, se aplicó una métrica de *silhouette* penalizado por ruido (1.2.6). De esta manera, se seleccionó la combinación $(\varepsilon^*, minPts^*)$ que maximiza dicho *score*, equilibrando la calidad de los clusters con la minimización del ruido. El detalle de la búsqueda puede consultarse en el Notebook (Sección de DBSCAN).

1.2.6. Métricas de rendimiento

- **Inercia (distorsión) en K-means**: $J = \sum_{i=1}^{N} \|x_i - \mu_{c_i}\|^2$, usada en la curva de "ganancias decrecientes" para elegir K.
- **Log-verosimilitud en GMM**: $\mathcal{L} = \sum_{i=1}^{N} \log\Big(\sum_{k=1}^{K} \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)\Big)$, monitorizada hasta convergencia del EM.
- **Silhouette penalizado en DBSCAN**: Definimos la fracción de ruido $f_{\text{noise}} = \frac{\#\{i:\, \text{label}_i = -1\}}{N}$, y el coeficiente de silhouette clásico sobre los puntos no–ruido, $\text{Sil}(X_{\neg\text{noise}}, \text{labels}_{\neg\text{noise}})$. Entonces la métrica penalizada es $S_{\alpha,\lambda} = \text{Sil}(X_{\neg\text{noise}}, \text{labels}_{\neg\text{noise}})\,(1 - f_{\text{noise}})^{\alpha} \;-\; \lambda\, f_{\text{noise}}$, con $\alpha = 1{,}0$ y $\lambda = 0{,}5$, utilizada para seleccionar $(\varepsilon^*, minPts^*)$ que maximiza $S_{\alpha,\lambda}$, equilibrando coherencia interna y cantidad de ruido.

1.3. Resultados

1.3.1. K-means

Con un umbral $\alpha = 0{,}02$, se seleccionó el primer k tal que $r_k = \frac{L(k-1)-L(k)}{L(1)-L(2)} < \alpha$, de manera de frenar justo antes de que la curva de inercia se aplane casi completamente.

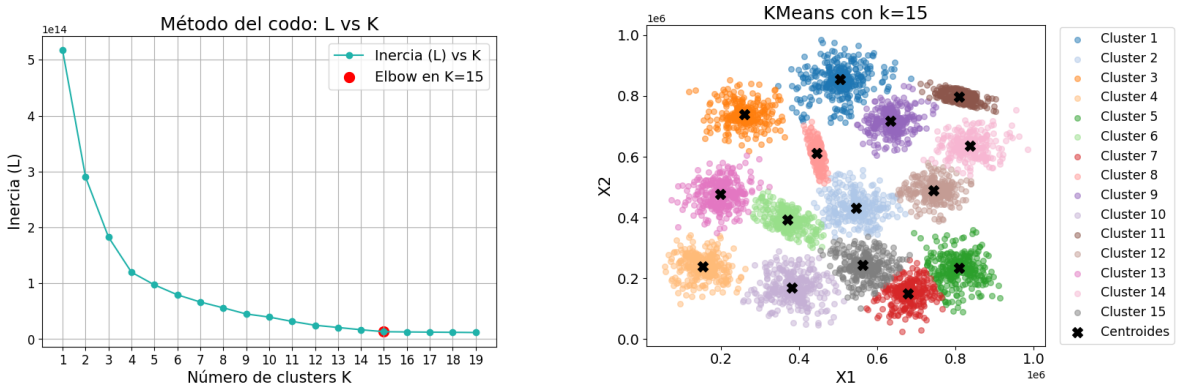
Aunque visualmente el "codo" más pronunciado aparece en $K = 4$ (véase Fig. 1a), este criterio cuantitativo permitió capturar la última mejora significativa antes de que se volviera constante, resultando en $K^* = 15$.

Al entrenar K-means con $K = 15$ se obtuvo una inercia final $L(15) = 1{,}3279 \times 10^{13}$ y el algoritmo convergió en 16 iteraciones.

En la Figura 1a se observa la evolución de la inercia junto al punto $\{K = 15\}$ elegido mediante el criterio de "ganancias decrecientes" anteriormente explicado (Sección 1.2.5). La Figura 1b muestra la división final en 15 clusters consistentes y bien diferenciados, cuyos centroides marcan regiones de alta densidad del conjunto de datos.

1.3.2. Gaussian Mixture Model (GMM)

Siguiendo la inicialización desde la ejecución previa de K-means con $K^* = 15$ y aplicando el algoritmo de Expectation–Maximization, el modelo GMM alcanzó una convergencia estable



(a) Inercia vs. número de clusters

(b) Segmentación final con $K = 15$

Figura 1: Evaluación mediante método del codo y resultado de K-means

con una log-verosimilitud final de $\mathcal{L}_{\text{final}} = -131948{,}36$ después de 100 iteraciones.

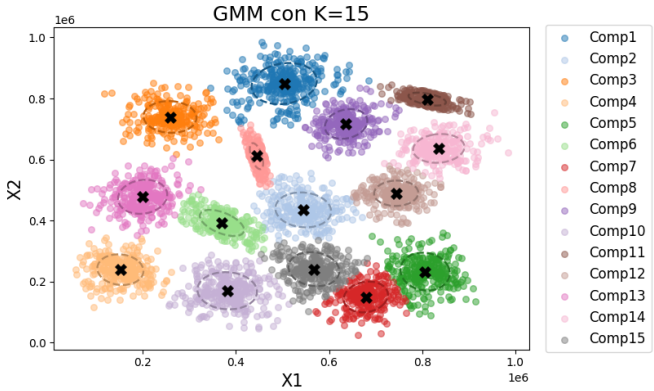


Figura 2: Asignación de clusters por GMM con $K = 15$, inicializado desde K-means

La Figura 2 muestra la asignación de cada punto a sus componentes gaussianas.

Aunque la partición resulta muy similar a la de K-means (Fig. 1b) dado que se partió de esos centroides, GMM ajusta las medias y estima además para cada componente una matriz de covarianza Σ_k, representada mediante "elipses de dispersión o covarianza". En la Tabla 1 en la Sección Apéndice (3) se comparan las coordenadas de los centroides de K-means con las medias ajustadas por GMM.

Las diferencias en las medias son mínimas (menores al 1 % en casi todos los clusters), lo que explica la similitud visual con K-means. El principal aporte de GMM son las elipses de covarianza: cada una está orientada según la dirección de máxima varianza del cluster y su tamaño refleja la dispersión de los puntos en cada eje. Esto permite modelar solapamientos y formas alargadas que K-means, con particiones rígidas de varianza uniforme, no puede capturar.

1.3.3. DBSCAN

A partir de la estrategia descrita en la Sección 1.2.5, donde se exploró una malla de $\varepsilon \in [0, 0{,}25]$ y $minPts \in [30, 200]$, la Figura 3 muestra dos visualizaciones superpuestas: a la izquierda, el número de clusters detectados; a la derecha, la cantidad de puntos etiquetados como ruido. En el extremo izquierdo (valores bajos de ε), predominan los puntos de ruido y quedan muy pocos clusters, mientras que en el extremo derecho (valores altos de ε con $minPts$ chico) el algoritmo agrupa casi todo en un único cluster con muy poco ruido. La franja intermedia marrón en la parte en la que concuerda con el azul más oscuro (menos ruido), es decir, alrededor de $\varepsilon \approx 0{,}2$ y $minPts \approx 100$, identifica un buen balance entre recuperar los 15 grupos reales y mantener un nivel de ruido moderado.

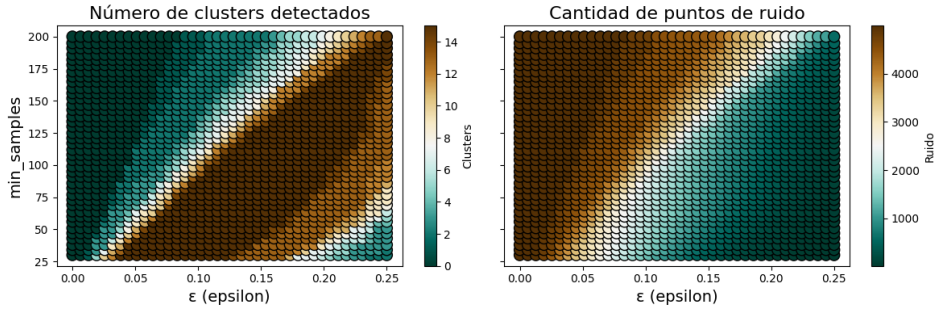


Figura 3: Barrido de hiperparámetros para DBSCAN: número de clusters (izquierda) y puntos de ruido (derecha).

Para hacer una selección más precisa, se utilizó el *silhouette score* penalizado por ruido (Ecuación 1.2.6) como se explicó anteriormente, resultando en

$$\varepsilon^* = 0{,}2308, \quad minPts^* = 130,$$

con 15 clusters, 266 puntos de ruido y un valor de $S_{1,0;0,5} = 0{,}597$. El gráfico de la Figura 4 muestra la clasificación obtenida, donde las clases se distinguen claramente, y las cruces negras indican los puntos considerados ruido.

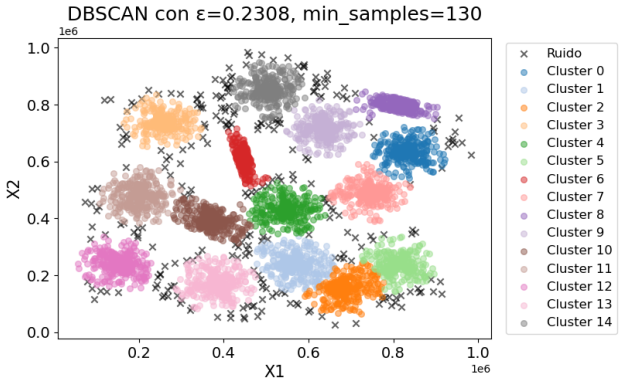


Figura 4: Resultado de DBSCAN con $\varepsilon = 0{,}2308$, $minPts = 130$; "$\times$" marca los puntos de ruido.

Este resultado confirma que la configuración elegida recupera la estructura de 15 grupos previamente identificada por K-means y GMM, al tiempo que descarta las observaciones menos densas como ruido.

En términos de eficiencia computacional y simplicidad, K-means clasifica correctamente en 15 grupos de manera rápida y con bajo costo, por lo que es adecuado cuando solo interesa la partición en clusters compactos. Si se necesita información adicional sobre la forma y dispersión de cada grupo, GMM aporta esas elipses de varianza a costa de un cómputo ligeramente más caro. Finalmente, si el objetivo incluye detectar y separar outliers, DBSCAN es la opción preferible, porque recupera los mismos 15 clusters mientras etiqueta explícitamente los puntos de ruido.

2. Reducción de dimensionalidad

2.1. Introducción

En este ejercicio se aborda el problema de reducción de dimensionalidad aplicado sobre el dataset `MNIST_dataset.csv`, en el cual cada imagen de dígito (del 0 al 9) de resolución original de 28×28 píxeles en escala de grises, se despliega como un vector aplanado de 784 valores, correspondientes a la intensidad de cada píxel. La alta dimensionalidad de estas representaciones impone no solo un gran costo computacional en tareas de procesamiento o modelado, sino que también puede dificultar la captación de las estructuras intrínsecas de los datos.

La entrada del algoritmo, por lo tanto, es la matriz de tamaño $N \times 784$ donde N es el número de muestras, y la salida es en una proyección de dichas muestras a un espacio de dimensión significativamente menor, determinado por el número de componentes principales retenidas, junto con la reconstrucción aproximada en el espacio original. Esta transformación se va a evaluar cuantificando el error cuadrático medio de reconstrucción en función de la cantidad de componentes usadas, y se va a comparar, para un conjunto de muestras de prueba, las imágenes originales versus sus reconstrucciones, mostrando la fidelidad de la reducción.

2.2. Métodos

2.2.1. Análisis de Componentes Principales (PCA)

El Análisis de Componentes Principales (PCA) busca proyectar los datos originales, representados por la matriz $X \in \mathbb{R}^{N \times D}$, sobre un subespacio de dimensión reducida $M \ll D$ de manera que la varianza explicada sea máxima. En la implementación llevada a cabo en el notebook (clase `PCA` en `models.py`), primero se calcula el vector de medias $\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i$ y se centra la matriz de datos $X_c = X - \mathbf{1}\,\bar{x}^T$. A continuación, se realiza la descomposición en valores singulares

$$X_c = U \,\mathrm{diag}(S)\, V^T,$$

donde U y V son matrices ortogonales y S contiene los valores singulares. Algebraicamente, esta factorización equivale a diagonalizar la matriz de covarianza empírica $C = \frac{1}{N} X_c^T X_c$, cuyos autovectores (las columnas de V) definen las direcciones principales del espacio y cuyos autovalores (proporcionales a S_i^2) cuantifican la varianza explicada por cada componente.

La proyección de un punto x al espacio reducido viene dada por

$$z = (x - \bar{x})\, V_M^T, \tag{5}$$

donde V_M toma las primeras M filas de V^T, es decir los autovectores asociados a los M mayores autovalores. Dado que V es ortogonal, esta multiplicación extrae las coordenadas de $(x-\bar{x})$ en esos ejes de máxima varianza; al truncar a las primeras M componentes, conservamos la parte de la señal con mayor energía informativa. La reconstrucción aproximada se expresa como

$$\hat{x} = z\, V_M + \bar{x}. \tag{6}$$

Para asegurar que los valores reconstruidos se mantengan en un rango válido cuando X representa, por ejemplo, intensidades de píxel, se recortan los resultados al intervalo $[0, 255]$, evitando valores negativos o mayores que 255.

Por último, la fracción de varianza acumulada explicada por las primeras k componentes se calcula como

$$\text{cumvar}_k \;=\; \frac{\sum_{i=1}^{k} S_i^2}{\sum_{j=1}^{D} S_j^2}, \tag{7}$$

lo que permite seleccionar M de manera que se supere un umbral de varianza retenida deseado.

2.2.2. Autoencoder Variacional (VAE)

El Autoencoder Variacional es un modelo generativo que combina una red *encoder* y otra *decoder* para aprender, en el espacio latente de dimensión L, una distribución aproximada de los datos. Dado un vector de entrada $x \in \mathbb{R}^D$, el encoder produce dos vectores $\mu(x)$ y $\log \sigma^2(x)$ que parametrizan una distribución gaussiana en el espacio latente. Gracias al "truco de reparametrización"

$$z = \mu(x) + \sigma(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

se preserva la diferenciabilidad y se puede hacer backpropagation. El decoder toma z y genera la reconstrucción $\hat{x}$. El entrenamiento minimiza la función de pérdida

$$\mathcal{L}(x, \hat{x}, \mu, \log \sigma^2) = \underbrace{-\mathbb{E}_{q(z|x)}[\log p(x|z)]}_{\text{reconstrucción}} \;+\; \underbrace{\mathrm{KL}\big(q(z|x) \,\|\, p(z)\big)}_{\text{regularización}}, \tag{8}$$

implementada como entropía cruzada binaria más divergencia de Kullback–Leibler. Se utilizó PyTorch para definir la clase `VAE`, entrenarla con Adam, y registrar las pérdidas de entrenamiento y validación por época, comparando finalmente la calidad de reconstrucción frente a PCA.

2.2.3. Conjunto de datos

Para ambos métodos se usó el dataset `MNIST_dataset.csv`, en el que cada fila corresponde a una imagen de dígito escrito a mano (0–9) representada como un vector de 784 valores (intensidades de píxel) precedido por su etiqueta. Después de cargar el CSV con pandas, se separó la primera columna como vector de etiquetas y y el resto como matriz de datos X. Para el entrenamiento del VAE, para cada valor de X se escaló dividiendo por 255 para normalizar las intensidades al rango $[0, 1]$. Además, para este caso, se convirtieron estas matrices en tensores de PyTorch y se extrajeron aleatoriamente 10 imágenes (sin reemplazo) para usar de conjunto de prueba (*test*) que estuvo completamente aislado de los procesos de entrenamiento y

validación. El resto de las muestras se dividió de manera aleatoria en un 90 % de entrenamiento y un 10 % de validación. Ambos subconjuntos (`train` y `val`) se cargaron mediante `DataLoader` con `batchsize=128` y `shuffle=True` únicamente en el conjunto de entrenamiento.

2.2.4. Hiperparámetros y validación

En el caso de PCA, la única decisión relevante fue la cantidad de componentes principales M a retener, la cual se eligió en función de la varianza acumulada explicada por los vectores singulares. Para el Autoencoder Variacional (VAE), realizamos una búsqueda sistemática mediante validación cruzada de 5 folds sobre el subconjunto *trainval*. En cada iteración de la búsqueda consideramos: latent_dim $\in \{2, 5, 10, 20, 50\}$, hidden_dim $\in \{200, 400, 800\}$, lr $\in \{10^{-2}, 10^{-3}, 10^{-4}\}$.

Para cada combinación (L, H, η) se dividió el conjunto *trainval* en 5 subconjuntos disjuntos de aproximadamente 1566 muestras cada uno. En cada fold, se entrenó un VAE con arquitectura input_dim $= 784$, hidden_dim $= H$, latent_dim $= L$, utilizando Adam con tasa de aprendizaje η, tamaño de batch 128 y entrenamiento durante 7 épocas. Se calculó la pérdida de validación en la última época de cada fold y se promedió sobre los 5 folds.

La combinación que minimizó la pérdida de validación promedio $\overline{\mathcal{L}}_{\text{val}} = \frac{1}{5}\sum_{f=1}^{5} \mathcal{L}_{\text{val}}^{(f)}$ fue elegida como (L^*, H^*, η^*). Con estos hiperparámetros óptimos, se reentrenó el VAE completo sobre *trainval* durante 20 épocas, usando el mismo optimizer y configuración de batch.

2.2.5. Métricas de rendimiento

Para cuantificar la calidad de la reducción y reconstrucción de los datos, se utilizaron las siguientes métricas:

- **Error cuadrático medio de reconstrucción (MSE).** Para PCA, y también al comparar frente a VAE sobre el mismo subconjunto de validación, se calculó

$$\text{MSE} = \frac{1}{N\,D}\sum_{i=1}^{N}\sum_{j=1}^{D}\big(x_{ij} - \hat{x}_{ij}\big)^2, \tag{9}$$

  donde x_{ij} es la intensidad real del píxel j en la muestra i y $\hat{x}_{ij}$ su reconstrucción.

- **Pérdida promedio de evidencia inferior ligada (ELBO).** Para el Autoencoder Variacional se midió el valor medio por muestra de la función de pérdida combinada 8, reportada en cada época tanto para entrenamiento como para validación.

2.3. Resultados

2.3.1. PCA

A continuación se presentan los resultados obtenidos al aplicar Análisis de Componentes Principales (PCA) sobre el dataset de dígitos, evaluando tanto el error de reconstrucción como la varianza explicada.

En la Figura 5 se muestra el error cuadrático medio de reconstrucción (MSE) de reconstrucción al proyectar y reconstruir las imágenes con diferentes cantidades de componentes principales. Se observa una rápida caída del error al incrementar las primeras componentes:

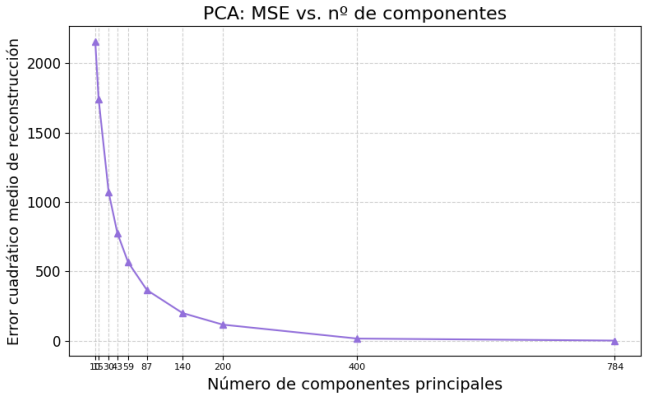


Figura 5: *Mean Squared Error* (MSE) en función del número de componentes principales.

vemos que con pocas variables latentes (por ejemplo, $k = 10$) el error es alto, pero luego el MSE ya disminuye drásticamente, y a partir de $k = 200$ el error se acerca a cero de forma casi asintótica.

La Figura 7(a) que se encuentra en el Apénice (3) despliega la varianza explicada de manera individual por cada una de las 784 componentes principales. Se aprecia claramente que las primeras componentes concentran la mayor parte de la varianza, mientras que, a partir de la componente ~ 100, la contribución de cada nuevo componente se vuelve muy chica. En la vista ampliada de las componentes 1–100 (Figura 7(b)), se evidencia que las primeras 20–40 componentes solas ya explican una fracción muy grande de la varianza total.

La Figura 8(a) que también se encuentra en el apéndice (3), muestra la varianza acumulada explicada al aumentar progresivamente el número de componentes principales. Las líneas horizontales y verticales (dibujadas en el gráfico original) indican los umbrales de 80 % (43 componentes principales), 85 % (59 componentes), 90 % (87 componentes) y 95 % (154 componentes). Esto evidencia que más allá de $k \approx 100$, el beneficio de agregar componentes adicionales es marginal tanto en varianza acumulada como en reducción del error de reconstrucción (Fig. 5).

En la Figura 8(b) se presentan ejemplos de reconstrucción de cinco dígitos de prueba (0, 5, 4, 1 y 9). En cada columna, se comparan la imagen original (fila superior) con sus reconstrucciones usando el número de componentes necesario para 80 %, 85 % y 90 % de varianza explicada. Se observa que la reconstrucción con $k = 43$ (80 % de varianza) ya captura la forma general de los dígitos, pero presenta mayor de borrosidad y pérdida de detalle fino. Al aumentar a $k = 59$ (85 %) la calidad mejora un poco más, y con $k = 87$ (90 %) las reconstrucciones casi coinciden con el original, manteniendo bordes nítidos y rasgos característicos de cada dígito, sin que se incremente demasiado el número de componentes principales.

De esta manera con $k = 87$, se alcanza un buen compromiso entre compresión y fidelidad: el MSE de reconstrucción es mucho más bajo en comparación con $k = 10$ (364.21 vs 2158.89 respectivamente), y las imágenes reconstruidas preservan los detalles esenciales de los dígitos (Fig. 8(b)).

Por lo tanto, para el entrenamiento final y las comparaciones posteriores, se eligieron $k = 87$ componentes principales, asegurando al menos el 90 % de la varianza explicada.

2.4. VAE

Para evaluar el rendimiento del VAE, utilizamos la mejor configuración hallada mediante validación cruzada, tal y como se explicó en 2.2.4:

$$(\text{latent_dim}, \text{hidden_dim}, \eta) = (20, 800, 10^{-3}),$$

con una pérdida de validación final promedio de 105,51. A continuación se comparan las reconstrucciones obtenidas por este VAE frente a las de PCA con $k = 87$ (90 % de varianza).

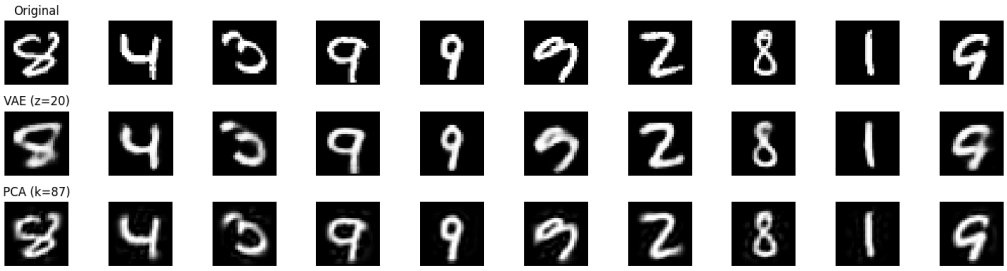


Figura 6: Comparación de reconstrucciones: fila superior (Original), fila intermedia (VAE con $z = 20$) y fila inferior (PCA con $k = 87$).

En la Figura 6 se observa que el VAE con espacio latente de dimensión 20 es capaz de captar la forma general de cada dígito y econstruir sus rasgos principales con mucha precisión. Frente a las reconstrucciones de PCA ($k = 87$), algunos de los resultados del VAE tienden a ser ligeramente más difusos en los detalles más finos, especialmente en trazos delgados o en bordes muy marcados (por ejemplo, en los dígitos "3" y "8"). Sin embargo, el VAE usa únicamente 20 variables latentes para codificar cada imagen, mientras que PCA requiere 87 componentes para alcanzar una calidad similar.

En términos cualitativos, el VAE con un espacio latente de dimensión 20 produce reconstrucciones globalmente coherentes que mantienen la legibilidad de los dígitos, aunque presentan un ligero borroneo en regiones de alto contraste. Esto se debe a que el VAE aprende una representación continua y regularizada, priorizando la generación de muestras verosímiles a costa de sacrificar algo de nitidez en los bordes. Por su parte, la reconstrucción mediante PCA con $k = 87$ tiende a conservar contornos más marcados y resulta visualmente más "nítida" en algunos casos. Sin embargo, para alcanzar esa calidad es necesario disponer de un espacio latente considerablemente más amplio.

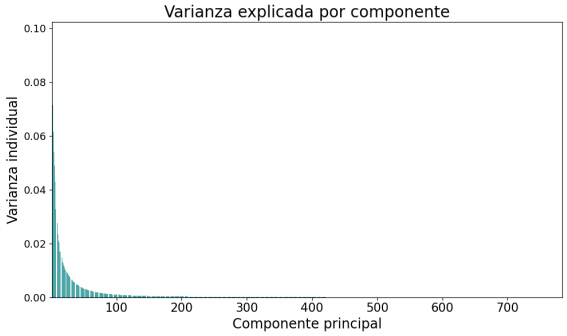
De manera cuantitativa, la pérdida de reconstrucción promedio en validación del VAE ($\approx$ 106,44) es comparable al error cuadrático medio obtenido con PCA para $k = 87$ (MSE $\approx$ 364,21), pero debe considerarse que el VAE optimiza una combinación de pérdida de reconstrucción binaria más un término de regularización KL, mientras que PCA minimiza exclusivamente MSE.

En conclusión, el VAE con $z = 20$ ofrece un balance muy favorable entre compresión y calidad de reconstrucción: logra resultados competitivos frente a PCA usando menos de un cuarto de las dimensiones latentes, a costa de introducir un suave efecto de difuminado en las estructuras más finas. Esto sugiere que, para tareas generativas o de representación donde importa el grado de condensación del espacio latente, el VAE es la opción preferible; sin embargo, si el objetivo es maximizar la fidelidad exacta de píxel, PCA con el 90 % de la varianza explicada resultaría en imágenes visualmente más nítidas.

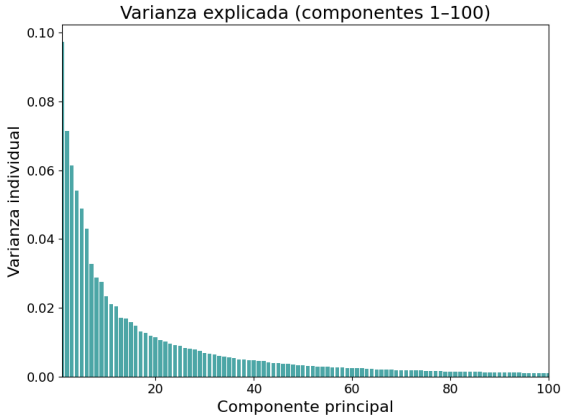
3. Apéndice

Cuadro 1: Centroides de K-means vs. medias de GMM ($K = 15$)

	K-means		GMM	
Cluster	X_1	X_2	μ_1	μ_2
1	503665	854338	503639.4	848998.6
2	371101	392427	370046.8	392366.5
3	380160	169091	381361.1	170139.2
4	545950	432863	544685.6	435068.0
5	562468	243230	567524.7	240980.8
6	809538	797406	810811.5	798129.8
7	677971	150751	679011.2	149685.8
8	742741	489417	744174.6	489608.7
9	258855	740048	258526.8	739614.5
10	808125	234833	806234.7	232888.8
11	633300	717099	636710.0	716633.6
12	444512	611892	444257.6	611222.2
13	152590	239488	151947.2	239269.1
14	199455	478149	199769.1	478031.6
15	836493	636549	835491.5	636631.8

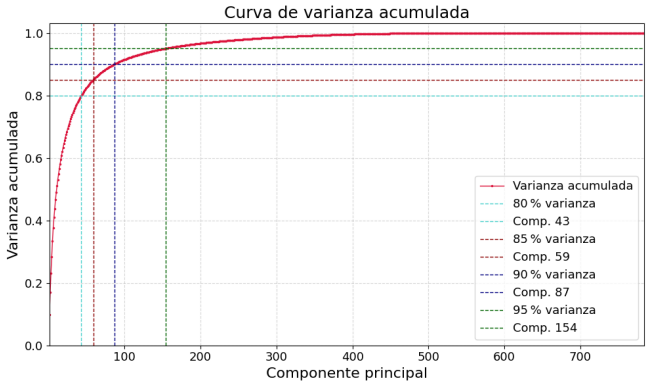


(a) Varianza explicada por cada componente principal (ejes 1–784).

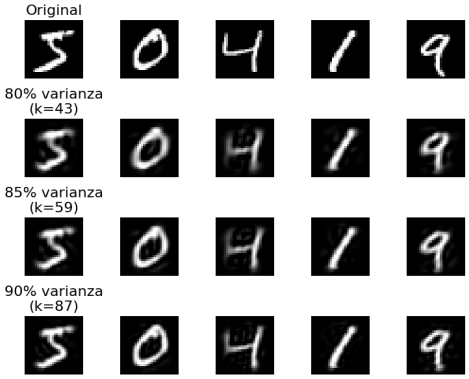


(b) Vista ampliada de las primeras 100 componentes.

Figura 7: Varianza individual explicada por cada componente principal.



(a) Curva de varianza acumulada explicada en función del número de componentes.



(b) Ejemplos de reconstrucción de dígitos con PCA: original y reconstrucciones para 80 %, 85 % y 90 % de varianza.

Figura 8: Análisis de varianza acumulada y ejemplos de reconstrucción con diferentes valores de k.